RateMyResume: Concurrency Load Test Report

Five simultaneous users, end to end: login, upload, text extraction, AI analysis, job-description matching. Prepared from a solutions-architecture perspective.

5 / 5	5m 46s	7m 15s	27 ms
users completed, 0 errors	average time to full result	slowest user (last in queue)	JD match, average

Executive summary

- **The application is functionally sound under concurrency.** All five users logged in, uploaded together, and received a score, an AI review, and ranked job matches. No request failed in this run and the test script recorded no errors.
- **The wait is dominated by local AI inference, not by the web application.** Login, upload, and JD matching together take well under a quarter of a second; the two language-model calls per resume account for essentially all of the elapsed time.
- **Users finish in a staircase.** The fastest got a result in 4m 23s, the slowest in 7m 15s, for the same document. Work is queued first-in-first-out onto a single laptop GPU.
- **The architecture is built to scale; the hardware is the limit.** The AI model, file storage, queue, and database each sit behind a replaceable interface, so moving inference to a GPU service and workers to several machines is configuration and deployment work, not a rewrite.

1. Test setup

Item	Detail
Scenario	5 Keycloak users (loadtest1 to loadtest5) log in, then upload the same PDF at the same moment. A script polls status every 2 s and times each stage.
Stack under test	Next.js frontend, FastAPI backend, PostgreSQL, Redis (queue and cache), Celery worker (thread pool, 2 concurrent jobs), Keycloak, Ollama (local LLM), PaddleOCR for scanned files.
Hardware	MacBook Air, Apple M3, 16 GB unified memory (from the Ollama log: about 10.7 GiB addressable by the GPU). Everything ran on this one machine.
Pipeline per resume	Extract text, then LLM call 1 (name, location, skills), then LLM call 2 (written review), then a deterministic rubric score; JD matching is a separate synchronous API call.
Runs compared	Run A: llama3.2:3b (from the Ollama log). Run B: model reported by the tester as llama3.2:1b. Run B raw data is in the appendix.

2. Results

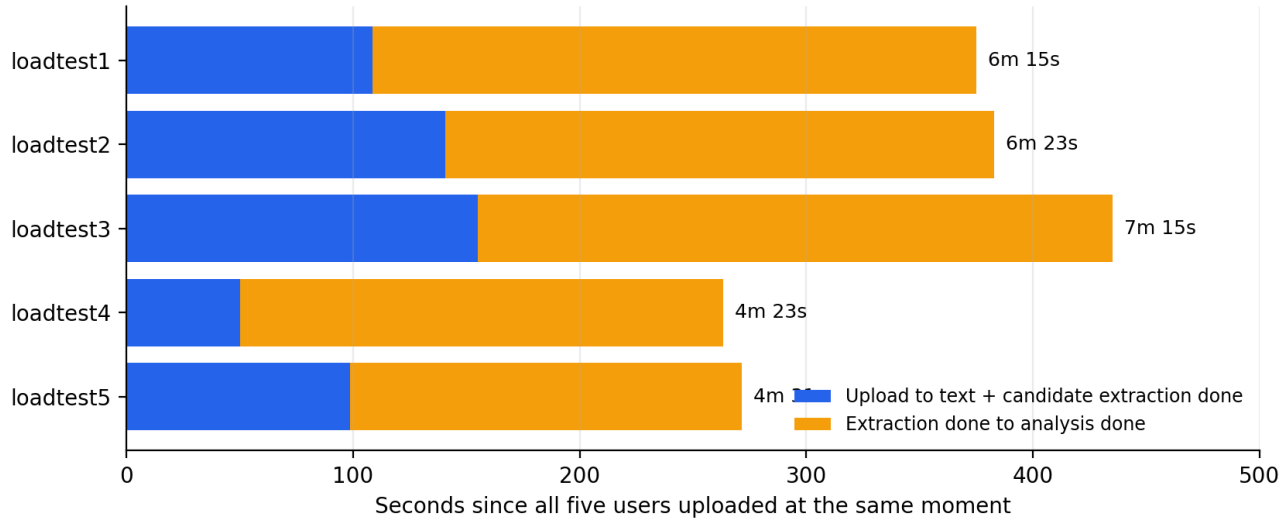


Figure 1. Per-user timeline, Run B. Each bar includes time spent waiting in the queue, not only processing time.

Extraction finished for the first user at 0m 50s and for the last at 2m 35s. Analysis then completed in three waves (about 265 s, 380 s, and 435 s). Pairs finishing within seconds of each other (users 4 and 5, then users 1 and 2) show the two worker slots running in parallel.

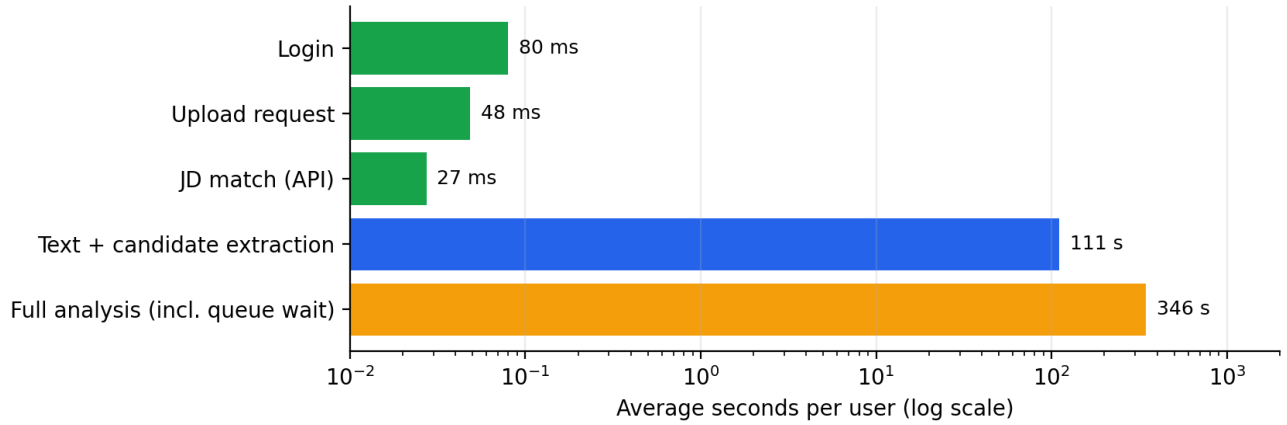


Figure 2. Average time per stage, log scale. The web tier is three to four orders of magnitude faster than the AI steps.

3. Model comparison

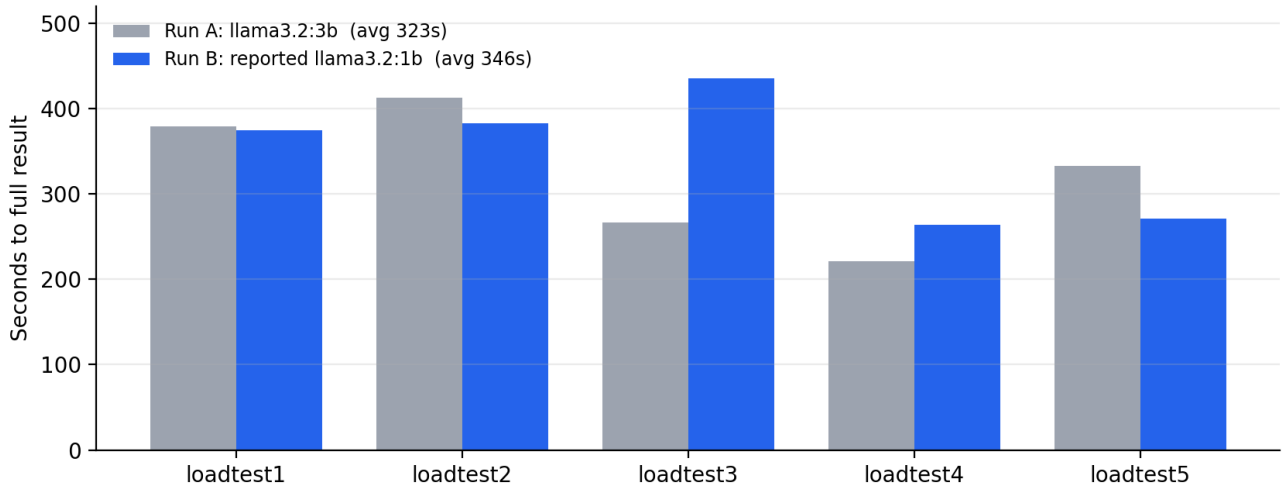


Figure 3. Time to full result per user, Run A versus Run B.

Metric	Run A (3b)	Run B (reported 1b)	Change
Avg extraction	94.5s	110.6s	+17.1%
Fastest extraction	58.6s	50.3s	-14.2%
Avg time to full result	322.6s	345.7s	+7.2%
Slowest (whole batch)	413.2s	435.2s	+5.3%
Throughput	44 resumes/h	41 resumes/h	-5.1%
Score / top JD match	88 / Full Stack (56%)	88 / Full Stack (56%)	identical

Reading this honestly: the smaller model did not make the pipeline faster; the batch was about 5% slower. A 1B model should generate noticeably faster than a 3B model, so this result points away from raw model size as the main cost. Two explanations fit the data and need one check each (see section 8): the request may still have been served by the 3B model, or time is going to something other than token generation, such as prompt processing, memory contention, or thermal throttling. The identical scores are expected: scoring and JD matching are deterministic rules, not model output.

4. Architecture and why it matters for scaling

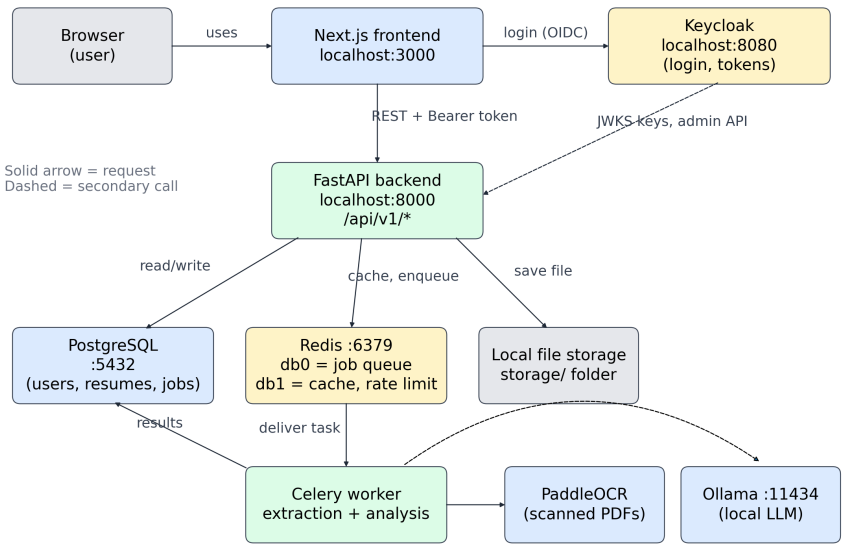


Figure 4. Runtime architecture as tested (all components on one laptop).

- **Asynchronous pipeline.** Upload returns in about 50 ms; slow work runs in Celery workers while the page polls status. The API never blocks on the model.
- **Database as source of truth.** Job state, attempt counts, and results live in PostgreSQL. Workers claim jobs atomically, so adding workers cannot double-process a job.
- **Resilience built in.** Transient model errors are retried with backoff; if the model keeps failing, extraction falls back to rule-based fields and analysis is marked failed without losing the resume. Redis cache and rate limiting fail open.
- **Replaceable parts.** The LLM sits behind a provider interface, file storage behind a storage interface, and the JD matcher is deterministic (about 27 ms), so it scales with the API at almost no cost.

5. Why it is slow on this machine

#	Finding	Evidence
1	Each resume needs two sequential LLM calls, and a single call is already slow on this hardware.	Before any parallelism, six consecutive Ollama calls took 46 s to 1m30s each. Extraction alone measured 43 to 57 s of worker time.
2	Parallel model requests share one GPU and one memory bus, so they do not scale linearly.	Two slots saved roughly a third versus an estimated fully serial run (no clean serial baseline was recorded), not half. Four slots caused timeouts.
3	Memory is tight on a 16 GB laptop running the whole stack.	With four parallel slots Ollama reserved about 7 GiB (3.5 GiB KV cache) and logged low-VRAM mode while about 6.5 GiB of system memory was free; Keycloak, PostgreSQL, Redis and OCR compete for the same pool.
4	A fixed 120 s request timeout met slower, shared inference.	With four slots, 13 requests failed with HTTP 500 after exactly 2m0s and five analysis jobs ended failed (LlmError) after three attempts. Raising the timeout and using two slots removed the failures.
5	First-in-first-out queueing creates a convoy.	Analysis jobs wait behind other users' extractions. The last user finished at 7m 15s although one resume needs roughly 100 s of model time.
6	A fanless laptop can throttle under sustained load.	Not measured here; the run-to-run spread is consistent with it. Repeat runs on mains power with temperatures logged would confirm.

6. Optimisation roadmap

Step	Change	Expected effect (estimate)	Effort
1	Confirm which model served requests; run on mains power, close heavy apps, repeat 3 times.	Trustworthy baseline; removes throttling noise.	Low
2	Run analysis jobs ahead of new extractions (queue priority).	Same total time, roughly a quarter lower average wait in a simple model of these timings.	Low-Med
3	Cut LLM work per resume: find skills with a dictionary, drop fields the review repeats, cap output length, tune context size.	Fewer generated tokens per call; generation is the main cost on this hardware.	Low-Med
4	Merge the two LLM calls into one structured call.	Up to about half of model time per resume; needs quality checks.	Medium
5	Separate queues and workers per stage; show queue position and ETA in the UI.	Predictable behaviour under mixed load; better perceived latency.	Medium
6	Move inference to a dedicated GPU service with request batching (for example vLLM or a hosted GPU), selected through the existing provider setting.	Largest gain: batching raises throughput well beyond a laptop; scales independently of the API.	Med-High

7. Scaling to production

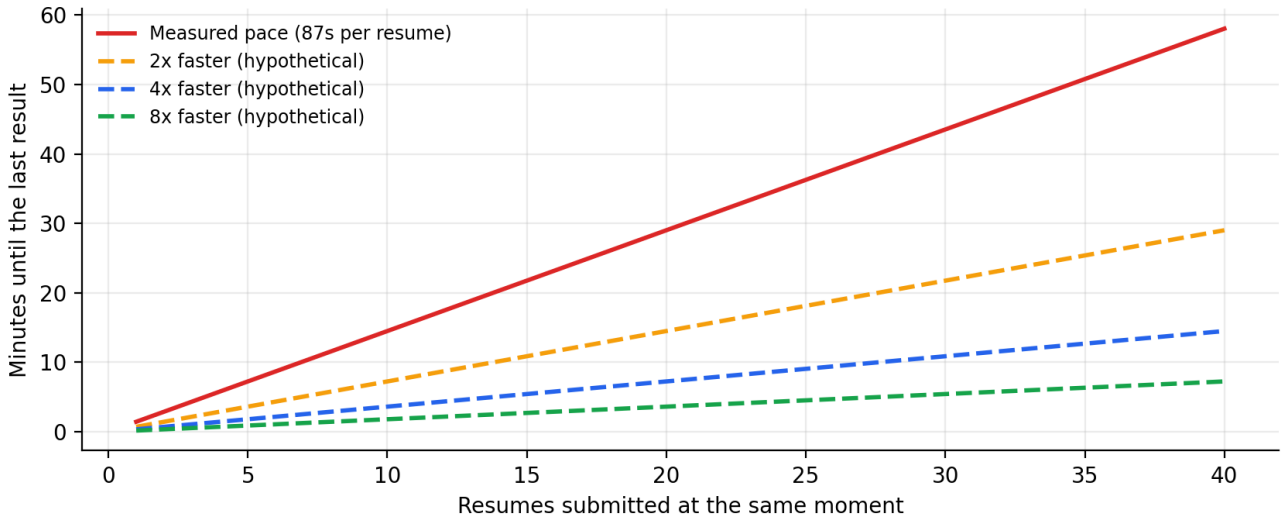


Figure 5. Projection, not a measurement: a linear model fitted to this run (about 87 s per resume). The faster lines are hypothetical what-if scenarios.

At the measured pace this laptop handles about 41 resumes per hour. Because the pace is nearly linear in queue length, capacity can be planned directly: total wait is roughly the number of waiting resumes times the per-resume model time divided by how many inference slots run in parallel. A target architecture follows from that:

- **Stateless API replicas** behind a load balancer; managed PostgreSQL and Redis; Keycloak in production mode with HTTPS.
- **Object storage** in place of the local storage folder, so API and workers on different machines see the same files.
- **Workers autoscaled on queue depth,** with extraction and analysis on separate queues.
- **A GPU inference tier** with batching, reached through the existing LLM provider interface. Resumes are personal data, so keep inference self-hosted or under a data-processing agreement.
- **Protection for the shared model:** per-user quotas (rate limiting already exists), upload limits, and a bounded queue that returns a clear message when full.
- **Observability:** queue depth, job latency p50/p95, tokens per second, failure and retry rates, and health checks.

8. Limitations and open items

- Single run per configuration, one document for all users, one machine; there is no variance estimate.
- Run A figures come from the console output of the script; Run B figures from its JSON file. Worker and Ollama settings for Run B are assumed unchanged apart from the model.
- **Verify the model.** Check `SELECT model_name, count(*) FROM resume_analyses GROUP BY 1;` to see which model the stored analyses recorded, and read tokens per second from the Ollama response fields (eval_count, eval_duration).
- Health checks were sampled only in Run A (207 of 207 healthy, slowest 0.11 s); they were not recorded for Run B.
- An earlier theory that expired login tokens caused script errors is not supported by this run: it lasted about 7 minutes with zero HTTP errors.

Appendix: Run B raw results (seconds)

User	Login	Upload	Extraction done	Analysis done	JD match	Score	Top match
loadtest1	0.09	0.05	108.6	375.0	0.026	88	Full Stack Developer (56%)
loadtest2	0.09	0.05	140.8	383.1	0.027	88	Full Stack Developer (56%)
loadtest3	0.08	0.05	154.9	435.2	0.028	88	Full Stack Developer (56%)
loadtest4	0.07	0.05	50.3	263.5	0.027	88	Full Stack Developer (56%)
loadtest5	0.07	0.05	98.6	271.5	0.028	88	Full Stack Developer (56%)